raglab

Un pipeline RAG local, mesuré brique par brique

Rapport technique

Architecture, protocole de mesure, phases d'industrialisation et résultats

Luc Melaye

Novembre 2026

Code : `https://github.com/VCLME/raglab` — licence MIT

Résumé

raglab est un système de *Retrieval-Augmented Generation* (RAG) qui tourne entièrement sur une machine locale : PostgreSQL et pgvector pour la recherche vectorielle, Neo4j pour un store graphe alternatif, Ollama (qwen2.5:7b) pour la génération, llama.cpp pour les modèles d'évaluation, FastAPI et React pour l'interface. Né comme laboratoire de comparaison des stratégies de découpage, il est transformé en pipeline prêt pour la production, avec une règle stricte : **une brique n'entre dans le pipeline que si elle apporte un gain mesuré** sur des jeux d'évaluation figés.

Ce rapport décrit l'architecture, le protocole de mesure (corpus indexés en entier, pertinence jugée au niveau du passage, questions juridiques typées, juges LLM d'une autre famille que le générateur et calibrés), les sous-phases d'industrialisation (9.1 à 9.8) et les résultats disponibles. Les chiffres de la baseline et des briques de retrieval sont en cours de mesure : ils apparaissent comme [à mesurer] et seront repris depuis les fichiers de résultats versionnés, jamais estimés.

Table des matières

Résumé **1**

1 Contexte et objectifs **3**
- 1.1 Le problème . . . 3
- 1.2 Objectifs . . . 3
- 1.3 Point de départ . . . 3

2 Architecture **3**
- 2.1 Vue d'ensemble . . . 3
- 2.2 Composants . . . 3
- 2.3 Indexation . . . 3
- 2.4 Chemin d'une requête . . . 4
- 2.5 Store graphe . . . 5

3 Méthodologie de mesure **5**
- 3.1 Principes . . . 5
- 3.2 Jeux d'évaluation . . . 6
- 3.3 Questions juridiques typées . . . 6
- 3.4 Juges RAGAS et calibration . . . 7

4 Les briques, phase par phase **7**
- 4.1 9.1 — Baseline reproductible . . . 8
- 4.2 9.2 — Recherche hybride BM25 + RRF . . . 8
- 4.3 9.3 — Re-ranking par cross-encoder . . . 8
- 4.4 9.4 — Routeur de requête et abstention . . . 9
- 4.5 9.5 — Durcissement de l'API . . . 9
- 4.6 9.6 à 9.8 . . . 9
- 4.7 Intégration continue . . . 9

5 Résultats **9**
5.1 Baseline et briques de retrieval 10
5.2 API : ce que l'utilisateur perçoit 10
5.3 Génération sur GPU 10

6 Limites et suite **11**
6.1 Limites connues 11
6.2 Suite 11

A Reproduire les mesures **11**

1 Contexte et objectifs

1.1 Le problème

Un RAG répond à une question en deux temps : il retrouve des passages pertinents dans un corpus, puis demande à un modèle de langage de rédiger une réponse appuyée sur ces passages. Chaque étape offre des dizaines de variantes : découpage des documents, modèle d'embedding, recherche dense ou lexicale, re-classement, prompt, modèle génératif. Dans la pratique, ces choix se font souvent à l'intuition, sur quelques questions de démonstration, et l'on ne sait pas dire si une brique ajoutée a amélioré quoi que ce soit.

1.2 Objectifs

1. **Tout en local.** Aucun appel à un service externe pour répondre : les documents restent sur la machine. Le budget matériel est celui d'un poste de travail, avec un GPU de 8 Go.
2. **Mesurer avant d'ajouter.** Chaque brique est comparée à une baseline sur les mêmes questions : Recall@5 et @10, MRR, latence p50 et p95, notes de juges RAGAS. Une brique qui n'apporte rien reste désactivée, derrière un drapeau.
3. **Un benchmark qui peut échouer.** Des jeux sur lesquels tout système obtient 100 % ne montrent ni gain ni régression ; les jeux de raglab sont construits pour être discriminants.
4. **Exploitable.** API avec streaming, budgets de temps, corrélation des logs, sondes de santé, intégration continue et porte de qualité qui bloque une régression.

1.3 Point de départ

À l'entrée de la phase 9, raglab savait ingérer des documents `.txt`, `.pdf` et `.docx`, les découper selon trois stratégies, les indexer dans pgvector ou Neo4j, générer des réponses citées et calculer Recall@k et MRR. Les jeux d'évaluation existants donnaient pourtant des scores flatteurs mais sans valeur : le corpus PIAF ne contenait que les paragraphes des questions tirées (6 paragraphes pour 30 questions, d'où un R@1 de 0,87), et chaque question LexML recopiait le titre du document attendu. La première tâche de la phase 9 a donc été de refaire la mesure.

2 Architecture

2.1 Vue d'ensemble

La figure 1 montre les composants. Les clients (interface web, CLI) passent par les mêmes services Python ; l'API n'ajoute aucune logique métier. Les deux modèles volumineux, le générateur servi par Ollama et le juge servi par llama.cpp, partagent un GPU de 8 Go et ne sont jamais chargés en même temps.

2.2 Composants

2.3 Indexation

Un document ingéré est découpé selon une stratégie, puis chaque morceau (*chunk*) est vectorisé :

— `traditional` : 900 caractères, chevauchement de 150 ;
— `slide` : fenêtre glissante de 900 caractères, pas de 300 ;

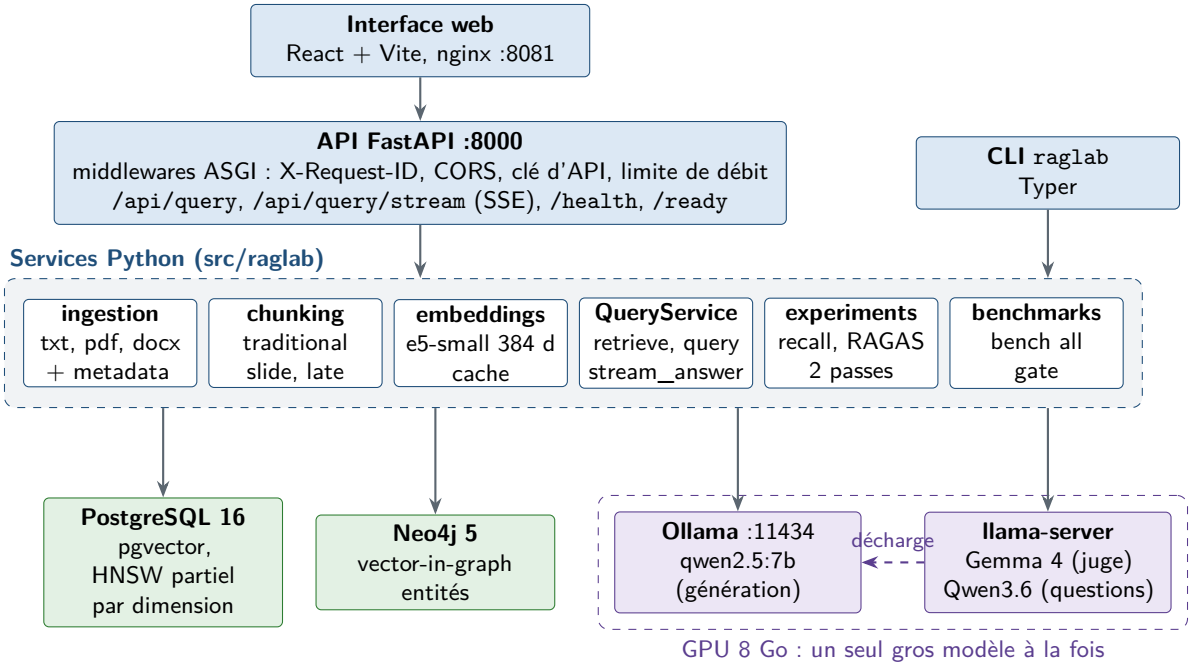


FIGURE 1 – Composants de raglab. Les flèches violettes en pointillé matérialisent l'orchestration du GPU par `LocalLLMServer` : avant un job llama.cpp, les modèles Ollama sont déchargés.

— `late` : chunk focal de 700 caractères dont l'embedding est calculé avec 500 caractères de contexte de part et d'autre.

Les vecteurs de dimensions différentes cohabitent dans une même colonne pgvector non dimensionnée ; un index HNSW partiel par dimension (`m = 16`, `ef_construction = 64`, distance cosinus) est créé à la volée. Chaque `index_run` pointe vers son espace d'embedding (fournisseur, modèle, dimension, préfixes `query:` et `passage:`) : la requête est toujours vectorisée avec le modèle de l'index interrogé.

2.4 Chemin d'une requête

1. L'API attribue un identifiant de requête (`X-Request-ID`, accepté ou généré), vérifie la clé d'API et la limite de débit si elles sont activées.
2. `QueryService.retrieve` résout l'index, vectorise la question (étape `embed`, budget de 30 s par défaut) puis cherche les k plus proches voisins dans pgvector (étape `retrieval`, sous `statement_timeout` de 15 s).
3. En streaming, l'événement SSE `retrieval` envoie immédiatement les sources, leurs scores et les temps par étape ; la génération Ollama suit, un événement `token` par fragment, puis `done` avec la réponse complète et le temps jusqu'au premier token.
4. Un échec est converti par une fonction unique en statut HTTP et en événement `error` : modèle absent (424), mémoire saturée (503 avec `Retry-After`), Ollama injoignable (502), délai dépassé (504 qui nomme l'étape).

Deux défauts trouvés en préparant la baseline ont été corrigés à cet endroit. **Le modèle d'embedding était rechargé à chaque requête**, avec des vérifications réseau auprès de Hugging Face : une requête RAG dépendait d'internet. Il est désormais gardé en mémoire, chargé une seule fois même sous concurrence. **Le top-k pouvait être incomplet** : l'index HNSW est partagé par tous les runs de même dimension et pgvector filtrait l'`index_run` après avoir lu 40 candidats. Avec plusieurs corpus en base, une question pouvait revenir avec moins de 5 chunks, voire aucun. La

Module	Rôle
`ingestion`	lecture et normalisation des fichiers, checksum, metadata par fichier compagnon JSON
`chunking`	trois stratégies interchangeables, gelées pendant la phase 9 (référence de comparaison)
`embeddings`	`intfloat/multilingual-e5-small` (384 dimensions) ou un modèle Ollama ; un modèle chargé par configuration
`indexing`	un `index_run` par corpus, stratégie et espace d'embedding ; mode incrémental
`retrieval`	résolution de l'index, embedding de la question, recherche KNN filtrée, génération
`generation`	prompt de *grounding* et citations `[Rank N]`, appel Ollama avec ou sans streaming
`graph`	store Neo4j : chunks en nœuds, index vectoriel par dimension, entités, recherche hybride
`experiments`	benchmark de recall, juges RAGAS, calibration des juges
`benchmarks`	jeux figés, `raglab bench all`, porte de qualité `raglab gate`
`llm`	client compatible OpenAI et `LocalLLMServer` (orchestration du GPU)
`api`	FastAPI, middlewares, correspondance erreurs vers statuts HTTP

TABLE 1 – Modules de `src/raglab`.

recherche itérative de pgvector 0.8 (`hnsw.iterative_scan = strict_order`) parcourt désormais le graphe jusqu'à remplir le top-k. Les deux correctifs sont désactivables pour que leur effet soit mesuré séparément (section 5).

2.5 Store graphe

Neo4j sert de backend alternatif, comparable sur les mêmes questions : chaque chunk devient un nœud portant son embedding, relié à son document et à son run ; un index vectoriel Neo4j par dimension sert la recherche KNN. Une extraction d'entités optionnelle (LLM ou spaCy) ajoute des nœuds `Entity` et un mode hybride combine score vectoriel et entités partagées. Sur l'ancien jeu PIAF réduit, pgvector et Neo4j donnaient des résultats identiques (mêmes chunks, mêmes embeddings) ; la comparaison sur les jeux refaits reste à faire.

3 Méthodologie de mesure

3.1 Principes

1. **Corpus complet.** On tire N questions, mais on cherche la réponse parmi *tous* les passages du corpus. Le nombre de questions et la taille du corpus sont indépendants.
2. **Tirage figé.** Questions triées par identifiant puis tirées avec une graine (42) : mêmes questions quel que soit l'ordre de la source. Les jeux sont versionnés dans `benchmarks/` et ne sont pas régénérés entre deux mesures.
3. **Pertinence au niveau du passage.** Chaque cas porte des `expected_spans` : fichier et positions de la réponse dans le texte normalisé. Un chunk est pertinent s'il chevauche ce passage. La vérité terrain ne dépend donc pas de la stratégie de découpage, et un chunk du bon document mais du mauvais endroit ne compte pas, ce qui est décisif pour des lois de plusieurs centaines de chunks.
4. **Métriques par type de question**, quand les cas sont typés.
5. **Traçabilité.** Chaque run écrit `benchmarks/results/<label>.json` : chiffres, machine, versions logicielles, empreintes des modèles et commande exacte.

Jeu	Langue	Questions	Corpus
PIAF	français	500	761 paragraphes de Wikipédia (split `train` complet, 540 Ko environ)
LexML texte intégral	portugais	277	252 lois fédérales brésiliennes, 4,6 millions de caractères, manifeste SHA-256
Abstention	fr + pt	80	aucune réponse dans les corpus (hors périmètre, quasi-sujet, conversationnel)

Table 2 – Jeux figés. Les deux premiers mesurent le retrieval et la génération ; le troisième, introduit pour la phase 9.4, mesure la capacité à refuser.

3.2 Jeux d'évaluation

PIAF. Questions rédigées par des annotateurs sur des paragraphes de Wikipédia. Les 500 réponses annotées ont été relocalisées dans le texte normalisé. Les annotateurs lisaient le paragraphe : les questions en reprennent souvent les mots, ce qui favorise la recherche lexicale par construction.

LexML texte intégral. Pour chaque année de 1988 à 2024, la liste des lois fédérales vient de l'API ouverte du Sénat brésilien ; on garde 7 lois par année dont le texte consolidé (planalto.gov.br) fait entre 3 000 et 150 000 caractères, passages révoqués retirés. Médiane : 8 600 caractères par loi. Le corpus n'est pas versionné, mais chaque fichier est vérifié contre le manifeste avant une mesure : si un texte a changé à la source, le run s'arrête au lieu de mesurer sur un autre corpus.

3.3 Questions juridiques typées

Les questions LexML sont écrites par un modèle rédacteur (Qwen3.6-35B-A3B, servi par llama.cpp en JSON contraint par grammaire, température 0) à partir de passages d'environ 700 caractères tirés dans le corps des lois. Trois types testent des capacités différentes :

Type	La question contient	Elle teste	Nombre
`paraphrase`	les idées du passage, sans numéro d'article ni de loi	la recherche sémantique	93
`reference`	le numéro de l'article et de la loi	les identifiants exacts, que les embeddings brouillent	97
`entity`	un sigle ou un nom propre du passage	les termes rares	87
Total utilisable (sur 168 lois)			277

Table 3 – Questions LexML par type.

Le modèle doit aussi rendre une réponse copiée mot pour mot du passage ; une question est rejetée si cette citation est introuvable, si la réponse est vide de sens (formule de promulgation, « VETADO »), si la question ne se comprend qu'à côté du passage (« neste trecho »), ou si son type n'est pas respecté. Un premier essai avec qwen2.5:7b a été abandonné : 187 rejets pour 147 questions gardées, et environ un tiers de questions vagues à la relecture. 40 questions ont été relues à la main ; les défauts trouvés sont devenus des règles automatiques, qui ont écarté 23 cas sur 300. Chaque cas enregistre aussi son recouvrement lexical avec le passage, pour vérifier plutôt que supposer l'avantage donné à la recherche lexicale.

3.4 Juges RAGAS et calibration

La qualité des réponses est notée par un modèle de langage (*LLM-as-judge*) sur trois critères : *faithfulness* (chaque affirmation est-elle appuyée par le contexte ?), *answer relevancy* (la réponse traite-t-elle la question ?) et *answer correctness* (dit-elle la même chose que la réponse attendue ?), sur une grille 0 / 0,5 / 1.

Le juge est **Gemma 4 26B-A4B**, d'une autre famille que le générateur : un modèle tend à préférer son propre style, et qwen2.5 ne doit pas noter ses propres réponses. Les 8 Go de VRAM ne contiennent pas les deux modèles ; la notation se fait donc en deux passes (figure 2), sans aucune intervention manuelle.

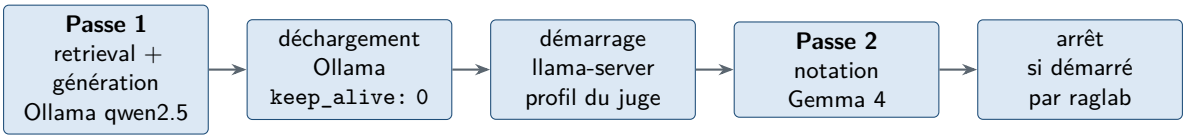


FIGURE 2 – Notation RAGAS en deux passes, orchestrée par `LocalLLMServer`.

Une note n'a de valeur que si le juge est fiable. La calibration fabrique, à partir de 30 paragraphes PIAF, des réponses dont on connaît le bon verdict : la réponse annotée (fidèle et pertinente), la réponse d'une autre question (inventée), la bonne réponse augmentée d'une affirmation absente du contexte (ajout inventé, le cas difficile), et la réponse à une autre question sur le même paragraphe (hors sujet). Une note compte comme positive à partir de 0,75. Le tableau 4 reprend les résultats de `benchmarks/judge_calibration.json`.

Taux de bons verdicts	**qwen2.5:7b (Ollama)**	**Gemma 4 26B-A4B (llama.cpp)**
Réponse fidèle reconnue fidèle	0,800	**0,900**
Réponse inventée repérée	**1,000**	0,967
Ajout inventé repéré	**1,000**	0,967
Faithfulness	0,933	**0,944**
Bonne réponse jugée pertinente	0,400	**0,967**
Réponse à une autre question repérée	**0,933**	0,700
Answer relevancy	0,667	**0,833**
Réponses notées / non lisibles	150 / 0	150 / 0

TABLE 4 – Calibration des juges (30 paragraphes, 5 réponses par paragraphe, graine 7).

Sur la fidélité, les deux juges se valent : choisir Gemma n'améliore pas la détection, il évite que le générateur se note lui-même. Sur la pertinence, qwen2.5 pénalise à tort plus de la moitié des bonnes réponses courtes. Gemma se laisse tromper 3 fois sur 10 par une réponse au bon format à une autre question ; c'est en partie le protocole, puisque le juge de pertinence ne voit pas le contexte. Un premier essai avec un seuil à 0,5 semblait montrer que les juges ne voyaient pas les ajouts inventés ; en réalité ils leur donnaient 0,5, la note juste selon leur grille. C'est le seuil qui était faux.

4 Les briques, phase par phase

La phase 9 transforme le laboratoire en pipeline de production. Chaque sous-phase fait l'objet d'une branche et d'une pull request avec un tableau avant / après ; rien n'est fusionné sans validation.

Phase	Statut	Contenu
9.1a Protocole	livré	jeux refaits, corpus complets, pertinence au passage, questions LexML typées, `LocalLLMServer`
9.1b Baseline	en cours	temps par étape, cache de l'embedder, scan HNSW itératif, `raglab bench all`, juges calibrés ; mesures en cours
9.2 Hybride	en cours	BM25 dans PostgreSQL et fusion RRF ; PR ouverte, première migration Alembic
9.3 Re-ranking	en cours	cross-encoder `bge-reranker-v2-m3` ; PR ouverte, désactivé par défaut
9.4 Routeur	en cours	compréhension de la requête et abstention ; jeu de 80 questions livré
9.5 API	livré	SSE, budgets par étape, request id, clé d'API, limite de débit, `/health` et `/ready`, erreurs typées
9.6 Observabilité	en cours	métriques Prometheus, tableaux Grafana, suivi de dérive
9.7 Portes de qualité	en cours	`raglab gate` et porte hebdomadaire en place ; seuils à fixer depuis la baseline
9.8 Vitrine	à venir	démonstration et documentation de présentation

TABLE 5 – Sous-phases de la phase 9 (production).

4.1 9.1 — Baseline reproductible

La baseline est mesurée en trois runs qui isolent chacun une cause :

- `baseline-v0` : le code d'avant la phase 9 (cache d'embedding et scan itératif désactivés) ;
- `baseline-v0-cache` : + modèle d'embedding chargé une fois ; isole le gain de latence, à retrieval identique ;
- `baseline-v1` : + scan HNSW itératif ; isole le gain de recall du correctif. C'est la baseline officielle.

Configuration : découpage `traditional`, e5-small sur CPU, pgvector cosinus, top-k 5 (10 récupérés pour Recall@10), qwen2.5:7b en Q4 sur GPU à température 0,1, juges RAGAS sur les 100 premières questions de chaque jeu.

4.2 9.2 — Recherche hybride BM25 + RRF

Les questions de type `reference` (« art. 49 da Lei nº 8.078 ») et `entity` (sigles) reposent sur des termes exacts que les embeddings denses rendent mal. La phase 9.2 ajoute une recherche lexicale BM25 calculée dans PostgreSQL ($k_1 = 1{,}2$, $b = 0{,}75$, statistiques tenues à jour à l'indexation) et fusionne les deux classements par *Reciprocal Rank Fusion* pondérée, sur 50 candidats de chaque côté :

$$\text{score}(d) \;=\; \frac{1-w}{k + r_{\text{dense}}(d)} \;+\; \frac{w}{k + r_{\text{lex}}(d)}, \qquad k = 60,\; w = 0{,}5.$$

Le mode (`dense`, `lexical`, `hybrid`), w et k sont exposés dans la CLI, l'API et l'interface. La mesure prévue croise les trois modes avec les deux jeux et les trois types de questions, puis fait varier w et k.

4.3 9.3 — Re-ranking par cross-encoder

Un cross-encoder lit la question et chaque passage ensemble, ce qui le rend plus précis qu'une similarité de vecteurs mais trop lent pour tout le corpus. Il re-classe donc les 40 premiers candidats et garde les k meilleurs. Modèle : `BAAI/bge-reranker-v2-m3` (multilingue, environ 1,1 Go de

VRAM en fp16), avec un cache de scores. L'option est désactivée par défaut tant que son gain n'est pas mesuré ; sa latence apparaît comme une étape séparée.

4.4 9.4 — Routeur de requête et abstention

Un RAG qui répond à tout invente quand le corpus ne contient pas la réponse. Le jeu d'abstention (80 questions : 27 hors périmètre, 23 « quasi-sujets » proches des corpus mais sans réponse, 30 conversationnelles) mesurera la capacité du système à dire qu'il ne sait pas, sans dégrader les réponses aux vraies questions.

4.5 9.5 — Durcissement de l'API

- `POST /api/query/stream` en SSE ; `POST /api/query` reste compatible.
- Budgets par étape (`embed`, `retrieval`, `generation`) réglables, 504 qui nomme l'étape.
- `X-Request-ID` propagé dans chaque ligne de log ; middlewares ASGI purs, qui ne mettent pas le flux en mémoire tampon.
- Clé d'API optionnelle, limite de débit par clé ou adresse, `/health` (vivacité) et `/ready` (Postgres, Ollama, modèle installé).

4.6 9.6 à 9.8

L'observabilité (9.6) ajoute des métriques Prometheus et des tableaux Grafana, avec un suivi de dérive des scores. Les portes de qualité (9.7) font échouer une commande, en local ou en CI, quand Recall@5, MRR ou la faithfulness passent sous un plancher versionné. La vitrine (9.8) clôt la phase.

4.7 Intégration continue

Chaque push et chaque pull request passent par `ruff` (lint et format), `mypy`, `pytest` (base et Ollama simulés, torch CPU), la vérification de types et le build du frontend, et le build des images Docker. Une porte hebdomadaire mesure le retrieval sur CPU avec une base pgvector de service et les corpus en cache ; la faithfulness, qui demande un GPU, se vérifie en local avec `raglab gate --run`.

5 Résultats

> Les valeurs [à mesurer] ne sont pas encore mesurées. Elles seront reprises de `benchmarks/results/<label>.json` (`raglab bench report`) ; aucune n'est estimée.

5.1 Baseline et briques de retrieval

	PIAF		LexML		Latence	
Run	**R@5**	**MRR**	**R@5**	**MRR**	**p50 / p95**	**Faithf.**
`baseline-v0`	[à mesurer]	[à mesurer]	[à mesurer]	[à mesurer]	[à mesurer]	[à mesurer]
`baseline-v0-cache`	[à mesurer]	[à mesurer]	[à mesurer]	[à mesurer]	[à mesurer]	—
`baseline-v1`	[à mesurer]	[à mesurer]	[à mesurer]	[à mesurer]	[à mesurer]	[à mesurer]
9.2 hybride	[à mesurer]	[à mesurer]	[à mesurer]	[à mesurer]	[à mesurer]	[à mesurer]
9.3 re-ranking	[à mesurer]	[à mesurer]	[à mesurer]	[à mesurer]	[à mesurer]	[à mesurer]

TABLE 6 – Recall@5, MRR, latence de la requête (retrieval) et faithfulness moyenne (juge Gemma 4) sur les jeux figés.

Mode (LexML, Recall@5)	`paraphrase`	`reference`	`entity`
dense (`baseline-v1`)	[à mesurer]	[à mesurer]	[à mesurer]
lexical	[à mesurer]	[à mesurer]	[à mesurer]
hybride	[à mesurer]	[à mesurer]	[à mesurer]

TABLE 7 – Recall@5 par type de question LexML. Hypothèse à vérifier : la recherche lexicale aide surtout les types `reference` et `entity`.

5.2 API : ce que l'utilisateur perçoit

Le streaming a été mesuré sans GPU, sur un serveur uvicorn réel dont la recherche et Ollama sont remplacés par des faux de vitesse connue (recherche 60 ms, préremplissage 300 ms, 120 tokens à 40 tokens/s, médiane de 10 exécutions) :

Mesure	**ms**
`/api/query` : réponse complète	3 381
`/api/query/stream` : sources affichables (événement `retrieval`)	**65**
`/api/query/stream` : premier token	**366**
`/api/query/stream` : fin (événement `done`)	3 484

TABLE 8 – Streaming SSE, sans GPU (`scripts/bench_api_hardening.py`).

L'utilisateur voit les sources environ 52 fois plus tôt et le début de la réponse environ 9 fois plus tôt ; le temps total augmente de 3 %. Les middlewares ajoutent environ 0,16 ms par requête. Le temps réel jusqu'au premier token sur GPU est [à mesurer].

5.3 Génération sur GPU

Avec qwen2.5:7b en Q4 sur une RTX 3070 Ti Laptop (8 Go), la génération passe d'environ 10,7 tokens/s sur CPU à 38 tokens/s sur GPU (× 3,6), pour environ 4,7 Gio de VRAM.

6 Limites et suite

6.1 Limites connues

— **Une seule réponse attendue par question.** Un autre passage qui répond aussi bien compte comme non pertinent : Recall et MRR sont des bornes basses.

— **Questions LexML écrites par un modèle.** Elles évaluent le retrieval sans favoriser le générateur, mais héritent des tournures du rédacteur.

— **Biais lexical de PIAF.** Les annotateurs lisaient le paragraphe ; le jeu favorise la recherche lexicale.

— **Calibration limitée.** 30 paragraphes et une graine : les écarts de quelques points ne sont pas significatifs. Calibration en français seulement ; les variantes fabriquées sont plus faciles à repérer que de vraies hallucinations.

— **Fonctionnel.** Pas d'OCR pour les PDF image ; ingestion et indexation synchrones côté API, sans file de jobs ; limite de débit en mémoire, propre à un processus.

6.2 Suite

1. Terminer la baseline (`baseline-v0`, `-v0-cache`, `-v1`) et en déduire les seuils de la porte de qualité.
2. Mesurer la recherche hybride et le re-ranking sur les deux jeux et par type de question ; ne les activer par défaut que si le gain est net.
3. Mesurer l'abstention et le routeur de requête (9.4), puis instrumenter le pipeline (9.6).
4. Mesurer le temps réel jusqu'au premier token sur GPU.

A Reproduire les mesures

Infrastructure, puis initialisation de la base :

```
docker compose up -d postgres ollama neo4j
docker compose -f docker-compose.yml -f docker-compose.gpu.yml up -d ollama
pip install -e .[dev]
raglab db init
```

Jeux d'évaluation (les questions sont versionnées ; ces commandes régénèrent les corpus) :

```
raglab bench download piaf --limit 500 --seed 42
raglab lexml fulltext --from 1988 --to 2024 --per-year 7 --seed 42
```

Mesures, comparaison et porte de qualité :

```
raglab bench all --label baseline-v1
raglab bench report baseline-v0 baseline-v0-cache baseline-v1
raglab gate --results benchmarks/results/baseline-v1.json
raglab experiment calibrate-judges --output benchmarks/judge_calibration.json
```

Le juge llama.cpp est démarré et arrêté automatiquement quand ces deux variables sont définies :

```
RAGLAB_LLAMACPP_START_COMMAND   # {profile} remplacé par le profil du juge
RAGLAB_LLAMACPP_STOP_COMMAND
```

Sinon, raglab utilise un serveur déjà lancé à l'adresse `RAGLAB_LLAMACPP_BASE_URL`.